

Rapport d'activité – Développement de l'application Laravel

1. Introduction

Le projet avait pour objectif de remplacer complètement le fichier Excel que le tuteur utilisait pour gérer les interventions et les factures. L'idée était de tout automatiser, d'avoir une gestion plus rapide, plus claire et moins sujette aux erreurs.

Pour réaliser ce projet, j'ai utilisé :

- Laravel 12 comme framework backend, pour structurer l'application et gérer l'API REST.
- Blade + Bootstrap pour le frontend, afin d'avoir une interface simple et réactive.
- MySQL pour la base de données, avec des migrations pour faciliter les modifications.
- Postman, Scribe et PHPUnit pour tester et documenter l'API.

Ce projet m'a permis de me plonger dans Laravel, de comprendre la logique des API, la gestion des rôles et de me familiariser avec les bonnes pratiques de développement.

2. Mise en place du projet et préparation

2.1 Analyse et conception

Avant même de commencer à coder, j'ai fait beaucoup de préparation avec mon maître de stage :

- On a discuté des besoins précis, des rôles (admin, technicien) et de ce qui était vraiment important à automatiser.
- J'ai rédigé un cahier des charges fonctionnel, listant les fonctionnalités essentielles et secondaires.
- J'ai dessiné le schéma de la base de données (MCD et MLD), pour visualiser toutes les tables et leurs relations.
- J'ai créé des cas d'utilisation pour chaque rôle afin de savoir comment l'utilisateur interagit avec le système.

2.2 Découverte de Laravel

Comme je n'avais jamais utilisé Laravel :

- J'ai suivi des tutoriels YouTube pour voir concrètement comment créer des migrations, des modèles, des relations, des routes...
- J'ai consulté la documentation officielle pour comprendre les concepts clés : migrations, relations entre modèles, authentification avec Sanctum et création d'API sécurisées.

Cette étape m'a permis de prendre en main Laravel rapidement, tout en appliquant directement ce que je voyais dans les tutos à mon projet.

2.3 Environnement technique

Pour le projet, j'ai choisi :

- Laravel 12 pour sa structure simple, sa sécurité et sa compatibilité avec les API REST « REpresentational State Transfer ».
- MySQL pour gérer les données avec fiabilité.
- Blade + Bootstrap pour créer un frontend simple et efficace.
- Scribe pour documenter l'API.
- Postman pour tester tous les endpoints.
- Architecture : backend en API REST, frontend en Blade.

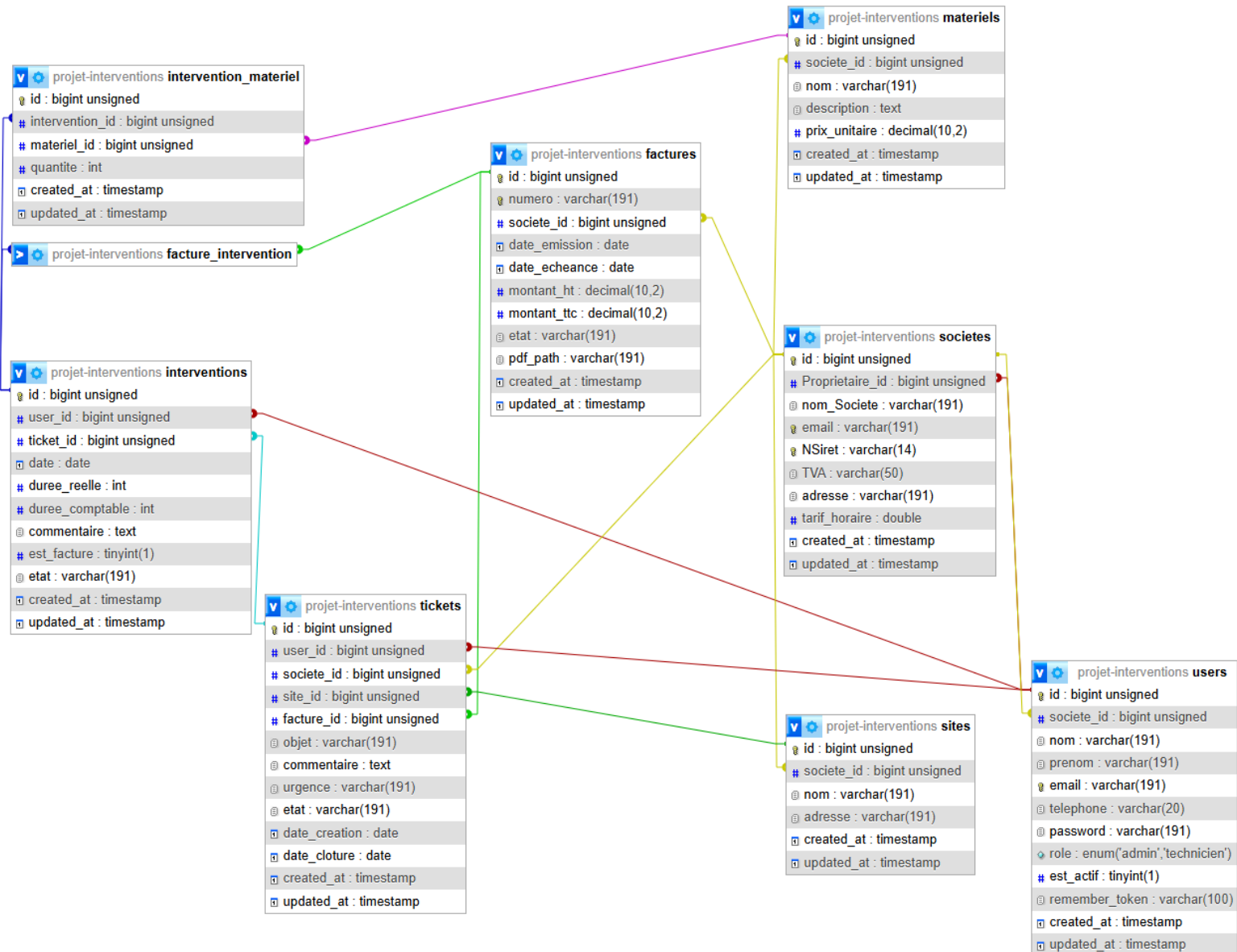
2.4 Structure initiale du projet

J'ai installé et configuré Laravel sur mon poste. Ensuite j'ai créé les migrations et modèles principaux : utilisateurs, sociétés, sites, tickets, interventions, matériel, factures. Enfin, j'ai défini les relations entre modèles (par exemple, un utilisateur appartient à une société, une société a plusieurs sites). Cette préparation m'a permis de poser des bases solides.

3. Développement des fonctionnalités

- Authentification avec Sanctum : login, logout, vérification du token.
- Gestion des utilisateurs : création d'un admin et de ses techniciens.
- Gestion des sociétés et sites : CRUD complet.
- Tickets et interventions : création, modification, suppression, ajout de matériel utilisé.
- Factures : génération automatique à partir des interventions non facturées

3.2 Modèle conceptuelle de la BDD



4. Sécurité et autorisations

- Contrôlers : Contrôler si les données sont bien dans le format voulu.
- Politiques : restreindre l'accès selon le rôle (admin/technicien).
- Validation des données via Form Requests pour sécuriser les input.

5. Documentation et tests

- Création de jeux d'essai avec le factories dans laravel
- Tests automatisés avec PHPUnit.
- Documentation API avec Scribe, accessible via /docs.
- Tests manuels avec Postman.

6. Résultats et utilisation

- API en partie fonctionnelle avec endpoints sécurisés.
- Base prête pour un frontend plus complet en Blade ou autre.
- Gain de temps et réduction des erreurs par rapport à Excel (->base de données prête).

7. Conclusion et perspectives

Le projet a transformé un processus manuel en une application centralisée, plus rapide et sécurisée.

Évolutions possibles :

- Dashboard plus complet
- finir le développement sur l'API -> (création,modifications,suppression... des informations dans la bdd)problèmes de droit/de redirection.
- Interface client pour créer des tickets.
- Module de géolocalisation pour calculer les frais de déplacement.
- Envoi automatique des factures par email